Raphael Production Blueprint and Runbook

Native Android companion + restricted laptop gateway

25 October 2026 | Revision 3.4 - Lean Production Readiness

Contents

- Start here: reviewed repository baseline and current risks
- Priority tiers and canonical build order
- R0 - Establish a reproducible baseline
- R1 - Correct Android generation state and SSE contract
- R2 - Define truthful fast/deep behavior
- R3 - Harden the restricted gateway and operations
- R4 - Android production hardening
- R5 - Real phone-to-laptop integration
- R6 - Privacy, reliability, and compact behavior evaluation
- R7 - Release candidate and operations handover
- Reference commands and useful files
- Official technical references

This is the lean implementation source of truth for taking the existing Raphael Android app and its restricted laptop gateway from the reviewed passing-CI baseline to a tested, secured personal sideload release. Revision 3.4 deliberately separates release-blocking correctness/security work from useful follow-up and optional engineering. Alexandria has its own companion runbook; this document owns Android, gateway, deployment, and end-to-end integration work.

> **Important:** these instructions describe required work. They do not claim that device testing, release signing, backup recovery, Play submission, or production deployment has already happened.

0. Start here

0.1 Repository baseline reviewed

- Repository: Agri99/Raphael
- Default branch: main
- Reviewed HEAD: 43dd995462963fb3191db95334a0e57dbc9bd982 (refactor: purge dead code shims, eliminate redundancy, and optimize route mappings)
- Prior implementation commit: 5cd536bc4b576bca5ac2b273321e75fb7aaebaf2 (align models, enforce CI cross-repo parity, add restore-db CLI and sync script)
- Latest Android CI observed: run 40089356662 - **success**
- Latest Gateway CI observed: run 40089356688 - **success**
- The Android workflow runs testDebugUnitTest and assembleDebug; it does not yet establish full-device coverage or release signing.

- At the reviewed baseline, the gateway workflow runs on Python 3.10, 3.11, and 3.12 and checks embedded/standalone Alexandria parity. Revision 3.4 calls for replacing the EOL 3.10 matrix entry after validation.

The repository is materially more complete than the prior review: the Gradle wrapper works, a gateway end-to-end cancellation test exists, the cross-repository checkout is in CI, the gateway has a restore CLI, and the embedded Alexandria files are synchronized at the reviewed baseline.

0.2 Current blockers and risks

The current default-branch heads were checked on 25 October 2026. Passing CI is a useful baseline, not proof of device behavior, restore recovery, or a production release.

Tier 1 - Must fix before a production claim

- **Retry selection:** `ChatViewModel.retryLast()` reads `lastOrNull()` from a DAO result ordered newest-first. Fix the order assumption and prove that Retry never removes the wrong user/assistant message.
- **Serialized stream persistence:** per-delta IO coroutines can write out of order or overwrite the terminal state. Use one serialized generation writer, bind the draft to its originating conversation, and make terminal transitions idempotent.
- **Release network policy:** the reviewed Android manifest allows cleartext globally. Disable it in release; any local HTTP exception must be debug-only.
- **Machine-specific endpoint:** `SettingsScreen.kt` has a private Tailscale hostname as a default. Remove it from tracked defaults. Deletion does not erase Git history; a hostname alone is not a credential, so do not rewrite history unless a real secret was exposed.
- **Memory restore complexity:** JSON restore duplicates the SQLite full-restore path. Remove `restore_json_snapshot` unless a concrete caller requires it; keep JSON export only if a human-readable export is useful.
- **Active-memory deduplication:** provider-instance locks do not enforce uniqueness across independent providers. After inspecting and resolving current active duplicates, enforce the invariant with a SQLite partial unique index.
- **Runtime support:** CI currently includes Python 3.10, which reached EOL in October 2026. Validate maintained Python versions and remove 3.10 from the supported matrix.
- **Real release evidence:** passing CI alone does not prove actual-phone/Tailscale behavior, successful SQLite recovery, or that the signing key is recoverable. Keep the compact phone matrix, one restore drill, and secure signing-key backup as hard release gates.

Tier 2 - Should improve, but keep compact

- Create shared raw SSE fixtures for `delta`, `done`, and `error`; test the same fixture bytes in the Python gateway and Kotlin parser.
- The checked-in `fast` and `deep` roles both use `qwen2.5:3b`; either select two installed models and measure the trade-off or label the longer-budget behavior honestly.
- Reconcile token-rotation documentation, README instructions, and `docs/DONE.md` with current code and evidence as the corresponding fixes are made.

Tier 3 - Could defer

- Major AGP/SDK/Kotlin toolchain upgrades are deferred unless device support, a security fix, or Google Play publication requires them. Replacing deprecated encrypted preferences is also a separate, tested migration - not an invitation to write custom cryptography.

0.2.1 Priority tiers - how to use this plan

- **Tier 1 - Must:** correctness, security boundaries, privacy, minimum runtime integration, recovery evidence, and stable release signing. These block the personal production/sideload release.
- **Tier 2 - Should:** clarity and measurement improvements that materially improve trustworthiness, maintainability, or regression detection. Finish when the Must path is green; a small, documented residual item does not automatically block sideloading if its fallback behavior is honest.
- **Tier 3 - Could:** large migrations and engineering enhancements with no current demonstrated requirement. Do not let these expand the release scope.

Each phase may contain actions from multiple tiers. A phase heading is not an instruction to implement every listed improvement before release.

0.3 Frozen architecture decisions

Do not change these choices unless measurements demonstrate a real need:

1. **Android client:** native Kotlin + Jetpack Compose, MVVM/StateFlow, Room for phone-local conversation history, DataStore for non-secret settings, OkHttp for SSE streaming.
2. **Inference gateway:** FastAPI in a separate Python process, bound to `127.0.0.1:8765`. It must remain separate from F.R.I.D.A.Y.'s desktop/voice process and must not import the desktop tool executor, computer-use stack, or unrelated startup side effects.
3. **Private network:** Tailscale Serve provides tailnet-only HTTPS to the loopback gateway. Do not use Tailscale Funnel, public port forwarding, or a public reverse proxy for this personal deployment.
4. **Authentication:** a high-entropy bearer token protects chat and memory routes in addition to tailnet access controls. Keep tokens out of Git, logs, screenshots, issue reports, and shell history.
5. **Model selection:** the phone sends only allowlisted aliases `fast` or `deep`; the server owns provider/model names and generation settings.
6. **Conversation privacy:** complete chat history remains in Android Room. The gateway receives only a bounded selection of completed turns for the active request; Alexandria stores concise, explicit memories, never transcripts.
7. **Memory lifecycle:** explicit, deliberate phone saves create approved memories immediately. Model-suggested memories are out of scope until a candidate/review workflow is separately designed.
8. **Persona:** the gateway owns the persona file; the Android app must not be the authority for persona or model rules.
9. **No premature expansion:** no embeddings/vector DB, automatic extraction, voice pipeline, on-device inference, generalized tool execution, or separate Alexandria service for this release.

1. Priority tiers and canonical build order

Keep one roadmap tree and complete it in this order. Do not start new UI/features while a Tier 1 invariant is failing.

1. **R0 - Baseline:** capture the real current SHAs, test output, and local model inventory. No blind resets.
2. **R1 - Correctness:** fix Retry, serialize stream writes, bind a draft to its conversation, make terminal transitions idempotent, and add shared SSE fixtures/tests.
3. **R3 - Boundary security:** remove the private hostname default, check token rotation/diagnostics/logging, retain loopback binding, and verify Tailscale Serve without Funnel.
4. **Alexandria A1-A4 - Data safety:** remove JSON restore, keep SQLite backup/restore, enforce active-memory dedup in SQLite, and verify migration/restore behavior.

5. **R4 - Android minimum production posture:** release cleartext disabled, secrets not logged/backed up, Room migration strategy, versioned release, stable signing-key backup. Avoid unrelated toolchain churn.
6. **R5 - Device reality:** run the full compact Xiaomi 12T / tailnet integration matrix against the exact candidate build.
7. **R2 and R6 - Behavior:** make labels honest; record small synthetic English/Indonesian checks; keep prompts, memories, and tokens out of logs.
8. **R7 - Release handover:** build/sign, install, verify rollback and backup recovery, record hashes and versions.

Tier 2 improvements may proceed after the corresponding Tier 1 invariant is green. Tier 3 work is explicitly outside the release path unless a concrete requirement appears.

2. R0 - Establish a reproducible baseline

Step 1 - Work from clean branches

In PowerShell, use your actual workspace paths. The examples assume `C:\Dev\Raphael` and a sibling `C:\Dev\Alexandria` because those locations are used by the current synchronization script; adjust once, then document your actual workspace.

```
Set-Location C:\Dev\Raphael
git status --short
git log -5 --oneline
```

Do not discard uncommitted changes. Commit or stash only after reviewing them yourself.

Step 2 - Run the Android baseline

Use JDK 17, which matches the checked-in CI. Run:

```
.\gradlew.bat --version
.\gradlew.bat testDebugUnitTest --no-daemon
.\gradlew.bat assembleDebug --no-daemon
```

Expected: all tasks complete with exit code zero and a debug APK appears under `app\build\outputs\apk\debug\`.

Step 3 - Run the gateway baseline

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
pytest -q
```

Use Python 3.12 for the local baseline unless your deployment is pinned to another supported version. The checked-in CI matrix should be updated to supported versions (proposed starting point: 3.11, 3.12, and 3.13 after dependency validation); Python 3.10 reached end-of-life in October 2026. Do not count a locally passing suite as a replacement for green GitHub CI.

Step 4 - Confirm installed Ollama models and settings

```
ollama list
python -m friday_mobile_gateway.cli status
```

Record installed tags, model file sizes, and the result of `status` in a local checklist that contains **no bearer token**. A configured name is valid only if the installed Ollama runtime can load that exact tag.

R0 acceptance gate

- ☐ Working tree status reviewed and preserved.
- ☐ Android unit tests and debug assembly pass from a clean checkout.
- ☐ Gateway tests pass locally on the documented supported runtime.
- ☐ Latest GitHub Android and Gateway CI are green for the commit being worked on.
- ☐ Python 3.10 is no longer presented as a maintained runtime after the replacement matrix has passed.
- ☐ Actual installed model tags are known; no assumptions are made from an old note.

3. R1 - Correct Android generation state before adding features

Primary files:

- app/src/main/java/com/agri/raphael/ui/chat/ChatViewModel.kt
- app/src/main/java/com/agri/raphael/data/local/Daos.kt
- app/src/main/java/com/agri/raphael/data/ConversationRepository.kt
- app/src/test/java/com/agri/raphael/ChatViewModelTest.kt
- app/src/main/java/com/agri/raphael/inference/RemoteInferenceBackend.kt

Step 1 - Fix Retry's descending-order assumption

`MessageDao.getRecentMessages()` returns newest-first (`ORDER BY createdAtEpochMs DESC`). `retryLast()` currently reads two messages and inspects `recent.lastOrNull()`, which is the older item in that returned pair. Make the selection order explicit. Prefer filtering for the newest non-complete assistant draft by `createdAtEpochMs DESC`, or deliberately sort to chronological order before selecting. Do not rely on an undocumented list order.

Required regression test:

1. Insert a completed user message.
2. Insert an interrupted assistant message with partial text.
3. Invoke `retryLast()`.
4. Assert only the interrupted assistant draft is removed/replaced; the user message remains.
5. Assert the new model payload contains completed turns and the current user request, but never the previous partial assistant draft.
6. Assert a complete assistant reply is never deleted by Retry.

If the desired product behavior is to retain interrupted drafts for inspection, do not delete them; instead mark them excluded from prompt context and create a new attempt. Choose one behavior and encode it in UI text and tests. The current implementation intends to delete the last incomplete assistant message, so the simplest compatible fix is to make that behavior correct.

Step 2 - Serialize streamed text and database persistence

Current `onEvent(Delta)` appends to a `StringBuilder` and launches an independent `viewModelScope.launch(ioDispatcher)` for each delta. Since the IO dispatcher is not a single ordered writer, writes can complete out of order. A stale `streaming` write can also race the final `complete/interrupted` write.

Implement one serialized generation state owner. A suitable design is:

- Create a generation-session object containing `conversationId`, `requestId`, `draftId`, the active `Call`, the accumulated text, and terminal state.
- Feed callback events to a single-consumer `Channel` or another serialized event pipeline.
- Append deltas in arrival order and batch UI/Room updates (for example, on a short interval or after a bounded character count); do not launch an uncoordinated Room write per token.
- Process exactly one terminal transition. After all queued deltas are applied, persist the final text and terminal status once.
- Make callbacks idempotent so cancel/error/EOF cannot complete the same generation twice.
- Do not call Room synchronously on the OkHttp callback thread.

Tests must include a fake backend that emits many deltas quickly, then completes. Assert persisted final text equals the exact concatenation, no chunk is missing/duplicated/reordered, and the status is `complete`. Add failure and cancellation variants; final state must not be overwritten by an earlier write.

Step 3 - Bind Stop and completion to the originating conversation

`stopGeneration()` currently searches recent messages using `_currentConversationId.value`. If the user navigates while generation is active, that value may no longer be the conversation where the active draft lives. Store the originating conversation ID and draft ID in the generation-session object and use those identifiers for every update. Either disable conversation switching during generation or deliberately support it with independent per-conversation state; do not implicitly use the currently visible conversation as a proxy.

Add tests for switching conversations while a generation runs, calling Stop, and receiving a late completion callback. The original draft must be updated; no unrelated conversation may be modified.

Step 4 - Enforce one active generation at the ViewModel boundary

The send action should be disabled in the UI during generation, and `sendUserMessage()` must also reject/reliably queue a second call while the ViewModel is generating. UI disablement alone is not a concurrency invariant. Define how a second rapid tap or repeated callback behaves and test it.

Step 5 - Model stream terminal states explicitly

The client must distinguish:

- `complete`: a valid terminal `done` event was received;
- `interrupted`: user canceled, transport disappeared, stream ended without `done`, or a terminal error occurred after partial text;
- request-level failure before a draft or response exists.

Keep partial text available for inspection, show a readable error, and offer Retry on interrupted output. A failed/partial assistant draft must never be added to a later model prompt. A Stop action must not mark a response complete merely because the user tapped Stop.

Step 6 - Share exact SSE fixtures between Python and Kotlin (Tier 1/2 regression protection)

Keep this small and protocol-focused. In the Raphael repository, create a fixture directory such as protocol-fixtures/sse/ (or use the repository's existing test-fixture convention if one already exists). Capture/construct fixtures from the actual gateway serializer and review them against the current protocol; do not invent a second, slightly different schema just for tests.

Minimum fixtures:

- delta: one user-visible text chunk with the exact event framing and JSON fields used by format_sse();
- done: the real terminal success shape;
- error: the real terminal failure shape and stable public error code;
- optionally malformed JSON, unknown event, and stream-ended-without-terminal examples as parser edge cases.

Load the same fixture bytes in Python gateway tests and Kotlin RemoteInferenceBackend parser tests. Do not copy/paste fixture content into two test files. Assert field parsing, newline/frame boundaries, UTF-8 content, error classification, and the rule that only a valid done marks a response complete. Keep fixture changes in the same pull request as any protocol change. This replaces the fragile practice of inferring client/server parity from unrelated tests; it does not replace the standalone Alexandria package parity check.

R1 acceptance gate

- ☐ Retry selects the correct message given the DAO's actual ordering.
- ☐ Stream deltas are persisted in order through a single writer.
- ☐ Terminal state is written after pending delta writes and cannot regress from complete to streaming.
- ☐ Stop, transport failure, error event, and unexpected EOF preserve partial text and mark the right draft interrupted.
- ☐ Conversation navigation cannot cause cross-conversation updates.
- ☐ Duplicate sends are prevented or explicitly queued.
- ☐ New regression tests pass in CI.
- ☐ Python and Kotlin tests consume the same reviewed delta/done/error SSE fixtures.

4. R2 - Configure and evaluate real fast/deep model roles

Primary files:

- src/friday_mobile_gateway/config.toml
- src/friday_mobile_gateway/config.py
- src/friday_mobile_gateway/role_map.py
- src/friday_mobile_gateway/provider_adapter.py
- src/friday_mobile_gateway/context_builder.py
- README.md
- docs/phase5_evaluation.md

- docs/DONE.md

Step 1 - Make the mode contract truthful before measuring it

Run ollama list; do not guess or silently download a bigger model. Then decide:

- fast: the smallest installed model that is responsive and acceptable for simple questions;
- deep: a model with measurably better multi-step coding/planning performance and acceptable laptop memory/latency.

The reviewed checked-in defaults use qwen2.5:3b for **both** aliases. The larger num_ctx and num_predict values make the deep request more expensive, but do not make the underlying model more capable. Choose one of two honest contracts: (A) configure two actually installed model tags, with deep showing a measured quality/latency benefit; or (B) keep one model and change the product copy/alias to something like standard and long-context (only if changing the public alias is feasible), or clearly disclose that "deep" means a larger generation budget rather than a stronger model. Do not download or hard-code a machine-specific model as part of the repo default. Confirm exact tags with ollama list.

Update TOML and Python fallback defaults together. Update README and evaluation metadata in the same commit. A local config override takes precedence over tracked defaults; load_settings() is evaluated at process import/start, so restart the gateway after changing configuration.

Step 2 - Keep role policy server-owned

- Client may send only fast or deep, never an arbitrary model name.
- Validate that the configured role model is not empty and that context/output limits are positive and sane.
- Keep the fast answer reserve and deep answer reserve separately configured.
- Enable thinking/reasoning mode only when the selected provider/model supports it and the stream parser test proves that only user-facing answer text is emitted. Do not assume that a thinking field and a content field can never arrive together.
- Handle missing model, unloaded model, model timeout, and Ollama offline with stable error codes and a Retry-capable client message.

Step 3 - Calibrate the context budget instead of treating the heuristic as fact

estimate_tokens() currently uses ceil(characters / 3). The Ollama adapter already logs prompt_eval_count on the final response chunk. Use a compact, repeatable set of 8-12 synthetic prompts:

- short English, long English, Indonesian, mixed English/Indonesian;
- emoji, code, punctuation-heavy text, long identifiers;
- memory-on and memory-off prompts;
- prompts near each configured context boundary.

For each case, record estimated prompt tokens, prompt_eval_count, response latency, and whether trimming preserved the current question. Do not store private chat text in logs; use synthetic fixture IDs and summarize aggregate results in the report. Adjust the estimate or safety margin so the gateway does not routinely underestimate the actual prompt. Preserve persona and current user message; trim oldest history first, then ordinary memories according to the existing policy. Never trim the system-owned persona.

R2 acceptance gate

- ☐ The UI, API alias, and documentation make no unsupported claim about model capability. **Tier 1 truthfulness gate.**
- ☐ README, TOML, Python defaults, and evaluation metadata agree.
- ☐ Missing/unavailable model behavior is covered by tests.
- ☐ Reasoning suppression is tested against the selected model's real stream shape.
- ☐ **Tier 2:** model/context choice is compared on a small synthetic English/Indonesian prompt set; TTFT and full response latency are recorded separately. A 20-case corpus and elaborate token calibration are not release blockers.

5. R3 - Harden the restricted gateway and operations

Primary files:

- src/friday_mobile_gateway/app.py
- src/friday_mobile_gateway/auth.py
- src/friday_mobile_gateway/schemas.py
- src/friday_mobile_gateway/routes/chat.py
- src/friday_mobile_gateway/routes/memory.py
- src/friday_mobile_gateway/streaming.py
- src/friday_mobile_gateway/observability.py
- src/friday_mobile_gateway/cli.py
- serve.ps1, serve.sh, README.md

Step 1 - Keep the boundary narrow

- Continue binding Uvicorn to 127.0.0.1:8765 only.
- Use tailscale serve --bg http://127.0.0.1:8765; never use tailscale funnel for this product.
- Do not add arbitrary file, shell, browser, computer-use, or F.R.I.D.A.Y. desktop-control routes.
- Keep /v1/health minimal and unauthenticated only if operationally useful; all data-bearing routes remain authenticated.
- Disable or protect /diagnostic outside development. It is not a production requirement.
- Keep /metrics authenticated and confirm it returns counts/latency/error codes only, not prompts, memories, token values, or reasoning.

Step 2 - Validate requests and errors

Add regression tests for:

- missing, malformed, wrong, and valid bearer token;
- malformed and negative Content-Length, declared oversize body, and streamed/chunked body exceeding the maximum even without a length header;
- invalid mode, invalid IDs, empty message list, excessive message count, oversized message, and malformed JSON;
- client disconnect during a response;
- provider exception messages containing quotes/newlines, ensuring every SSE error frame remains valid JSON and does not leak raw internals;

- queue full/timeout and cancellation while a request is active;
- no duplicate completion event, and no success metric for a canceled/failed request.

Keep public errors stable and concise. Avoid manually interpolating exception messages into JSON; use the shared format_sse()/json.dumps() path or a typed error serializer. Log a request ID and safe error code, not the request body, prompt, memory body, bearer token, or model thinking.

Step 3 - Check config loading and token rotation

rotate-token writes src/friday_mobile_gateway/config.local.toml and prints the new token once. The README must say this accurately. Check that:

```
git check-ignore -v src/friday_mobile_gateway/config.local.toml
git status --short
```

Local config must remain ignored and must never be committed. Apply user-only filesystem permissions to the file on Windows and do not paste tokens into GitHub issues or screenshots. Test token rotation end-to-end: old token returns 401, new token succeeds, and Raphael's stored token can be updated. A new token should not be copied into logs.

Step 4 - Improve the local operational contract

- Make the database path absolute at startup and show its resolved value only in local operator output, not public HTTP responses.
- Add explicit connect/read/overall timeout settings and a maximum model generation duration; test both normal and slow generation.
- Ensure gateway shutdown stops accepting work, cancels active generations safely, and releases the scheduler slot.
- Add tests for the body-limit middleware's receive/response behavior under disconnect and chunked uploads.
- Ensure the diagnostic endpoint and local development HTTP mode cannot accidentally be enabled in production.
- Update docs/DONE.md to represent verified facts, not stale test counts or manual checks that have not been performed.

Step 5 - Make Windows startup/restart deterministic

Use the gateway as a separate process from F.R.I.D.A.Y.; do not rely on a desktop assistant process being open. For the personal Windows deployment, choose one startup mechanism (for example Windows Task Scheduler configured to run at logon/startup with restart-on-failure) and document its exact command, working directory, Python environment, log location, and stop procedure. Keep serve.ps1 as Tailscale Serve configuration only if that is its documented purpose; it does not start Uvicorn. Avoid launching multiple gateway copies against the same database.

Minimum manual smoke sequence:

1. Start Ollama and verify the selected model exists.
2. Start the gateway once, bound to loopback.
3. Verify GET http://127.0.0.1:8765/v1/health.
4. Verify tailscale serve status and that the URL is HTTPS and tailnet-only.
5. Confirm local gateway ports are not listening on 0.0.0.0 or a LAN interface.
6. Test authenticated /v1/info and chat from a second tailnet device.

7. Stop and restart the gateway; verify the database and memory API still work.

R3 acceptance gate

- ☐ Auth, schema, size-limit, queue, error-frame, and cancellation cases are covered.
- ☐ Diagnostic page is not available in production unless explicitly authorized.
- ☐ No prompt, transcript, memory body, bearer token, or model reasoning enters logs.
- ☐ Token rotation is tested with both old and new tokens.
- ☐ One documented Windows startup method works after reboot and does not spawn duplicate instances.
- ☐ Gateway remains loopback-bound and Tailscale Serve is used without Funnel.

6. R4 - Android production hardening

Primary files:

- app/src/main/AndroidManifest.xml
- app/build.gradle.kts
- build.gradle.kts
- gradle/libs.versions.toml
- gradle/wrapper/gradle-wrapper.properties
- app/src/main/java/com/agri/raphael/security/TokenStore.kt
- app/src/main/java/com/agri/raphael/ui/settings/SettingsScreen.kt
- app/src/main/java/com/agri/raphael/settings/PreferencesStore.kt
- app/src/main/java/com/agri/raphael/data/local/RaphaelDatabase.kt
- app/src/main/java/com/agri/raphael/data/local/Daos.kt

Step 1 - Remove machine-specific defaults and protect the token field

- Remove the machine-specific Tailscale URL from SettingsScreen.kt and any related tracked default; use an empty or generic placeholder and explicit user configuration. Deleting it from the current commit will not purge public Git history. A hostname alone is not a credential; do not rewrite history unless a secret or materially sensitive endpoint was exposed. Consider renaming the machine only if it makes the old hostname stale.
- Make the bearer token field a password field (masked by default); provide a deliberate reveal toggle only if useful.
- Do not report that a connection is verified just because settings were saved. Distinguish “saved” from a successful health + authenticated info probe.
- Give connection probes explicit connect/read timeouts and display actionable errors without token values.
- Ensure token erasure removes the stored ciphertext and any derived local state. Test app restart after erase.

Step 2 - Make release networking secure by default

AndroidManifest.xml currently sets android:usesCleartextTraffic="true" at the application level. Tailscale Serve production traffic is HTTPS, so release builds must not allow arbitrary cleartext traffic.

1. Remove the global cleartext allowance from the main manifest.
2. If local HTTP development testing is still needed, enable it only in the debug source set or a debug-only Network Security Config.

3. Keep release networking HTTPS-only. Do not add certificate pinning by default for Tailscale Serve without a documented rotation/recovery plan; ordinary verified TLS plus tailnet access controls are the baseline.
4. Test that release builds reject `http://` URLs, debug builds behave as documented, and HTTPS works on the real phone.

Android official reference: Network Security Configuration.

Step 3 - Handle the existing deprecated secret store safely (Tier 2)

`TokenStore.kt` currently uses AndroidX `EncryptedSharedPreferences`, which Android documents as deprecated. Deprecation alone is not a reason to replace it with a hand-written Keystore wrapper during the Must release pass.

For this release:

- Keep the existing reviewed implementation unless it fails on the target device or a clear security issue is found.
- Confirm the encrypted preferences file is excluded from Android Auto Backup, because a restored preference file may be unusable if the original Keystore key is absent.
- Test save, read after process restart, disconnect/erase, and the behavior when decrypting stored state fails. Failure must ask the user to re-enter the token; it must not crash-loop or log the secret.
- Ensure the token is masked in UI and never included in logs, saved UI state, analytics, or crash strings.

Plan a migration only when there is a supported implementation and a testable migration path. If replacing the deprecated library later, prefer a maintained and reviewed crypto/storage solution (for example a vetted Tink-based design) rather than inventing encryption. Migrate old values safely, verify erase behavior, and make the change a separate commit. This is **Tier 2**, not a reason to block a personal sideload release if current behavior works and backup exclusion is verified.

Step 4 - Defer major toolchain/API upgrades unless a real requirement triggers them (Tier 3)

The reviewed app uses `compileSdk = 34`, `targetSdk = 34`, and Android Gradle Plugin 8.4.2. This is not automatically a blocker for a private APK sideloaded to your own phone. Google Play publication has separate current target-API rules; if Play distribution becomes a goal, re-check the live requirements and upgrade the compatible toolchain as one tested batch.

Do not upgrade AGP, Gradle, Kotlin/Compose, compile SDK, target SDK, and test dependencies merely to make the roadmap look more modern. Trigger this work only if:

- a required device/API behavior cannot be supported on the current toolchain;
- a security fix or required dependency forces the change; or
- you decide to distribute through Google Play.

When triggered, choose mutually compatible stable versions from the official compatibility tables; update AGP/Gradle/Kotlin/Compose as a coherent batch; then run unit tests, lint, debug/release assembly, and device tests. Do not infer release readiness from compile success alone. Keep the resulting version set and upgrade reason documented.

Step 5 - Configure production persistence and migration tests

- Set `RaphaelDatabase` to export Room schema (`exportSchema = true`) and check in the schema snapshots.

- Add an explicit migration strategy before any production schema version change. Do not use destructive migration for chat history.
- Test upgrade from every released schema version, empty install, app process restart, and database recovery behavior.
- Document privacy/retention behavior for chat history. `android:allowBackup="false"` is currently set and should remain deliberate; check the final merged manifest and backup rules as part of release QA.

Step 6 - Prepare release signing and build variants

- Add a `release` signing configuration sourced from protected local secrets or GitHub Actions secrets; never commit keystore files or passwords.
- Keep debug suffix and debug network allowances out of the release artifact.
- Decide whether minification/R8 is enabled; if enabled, test the signed, minified build, not just debug.
- Set a release versioning policy: increment `versionCode` monotonically; maintain a human-readable version.
- Produce and verify a signed APK/AAB. Record artifact SHA-256, source commit, tool versions, test results, and installation result.
- For Play distribution, prepare the required store listing, app privacy disclosures, and policy declarations truthfully. Do not claim the app transmits no data at all: prompts are sent to the user-operated laptop gateway for inference, while complete transcript history is not synced as a database.

R4 acceptance gate

- ☐ Release build disallows arbitrary cleartext HTTP.
- ☐ No committed private tailnet URL remains in defaults.
- ☐ Bearer token is masked, not logged, excluded from backup as required by the current encrypted store, and erasable; failure to decrypt is recoverable.
- ☐ Room schema export and non-destructive migration tests are enabled.
- ☐ Toolchain versions are recorded; a major SDK upgrade is performed only if triggered by distribution/support needs.
- ☐ Release artifact is signed with secrets outside Git, installed as a release build, and the stable signing key is backed up securely.

7. R5 - Real phone-to-laptop integration

This stage cannot be proven by GitHub CI alone. Run it with the gateway, Ollama, Tailscale, and Xiaomi 12T active.

Test setup

1. Confirm the laptop has the selected models installed (`ollama list`).
2. Start Ollama and the restricted gateway process.
3. Verify `tailscale serve status`; do not enable Funnel.
4. In Raphael Settings, enter the correct HTTPS Tailscale Serve URL and bearer token; save, then use **Test Connection**.
5. Install the current debug APK on the Xiaomi 12T. Repeat the full sequence with a release build later.

Integration matrix

Case	Procedure	Pass condition
Cold start	Start app after force-stop; open conversation.	App opens without crash; local history loads.
Fast mode	Send a short factual prompt.	Chunks appear progressively; terminal state is complete.
Deep mode	Send a multi-step code/planning prompt.	Correct model alias runs; response stays within budget; only answer text is displayed.
Retry	Interrupt a response, then retry.	Partial draft is excluded from prompt; current user request is retried once; no unrelated message is deleted.
Stop	Stop after several deltas.	Generation ceases; partial text remains marked interrupted; scheduler slot becomes available.
Bad token	Configure wrong token and probe.	Health may be reachable; authenticated route fails with 401 and reveals no secret.
Wi-Fi to mobile	Start generation, switch network, observe outcome.	Stream either completes or becomes visibly interrupted; no silent duplicate generation; Retry works.
Laptop sleep/wake	Sleep laptop during generation, then wake it.	UI reports interruption/reachability state; new requests work after gateway recovery.
Gateway restart	Stop/restart gateway during and between requests.	No stuck UI spinner; new requests work; memory DB remains valid.
Memory CRUD	Create, edit, delete, undo a memory.	UI and server agree; duplicate/idempotency errors are understandable.
Secret rotation	Rotate token on gateway, update phone setting.	Old token fails, new token succeeds; no token is logged.
Private boundary	Inspect listening address and Tailscale status.	Gateway listens only on loopback; only intended tailnet users can reach Serve.

Save a test report with timestamp, app version, commit SHA, Android OS version, model tags, network path and pass/fail. Do not attach real prompts, memory bodies, bearer tokens, or raw logs containing private content.

R5 acceptance gate

All rows pass on an actual device for the release candidate. Any failed case becomes a reproducible issue with a synthetic test prompt and expected behavior.

8. R6 - Privacy, reliability, and behavior evaluation

Step 1 - Keep a compact synthetic behavior/evaluation set

The current docs/phase5_evaluation.md is historical evidence, not a current acceptance result. Re-run it after the model/mode contract is chosen and label unit-level, local-runtime, and actual-device observations separately. Do not expand it into a large benchmark project before the release path works.

Use 8-12 synthetic cases that cover:

- answer without irrelevant memory injection;
- accurate recall of explicit stored memory;
- deleted memory exclusion;
- edited memory update;
- user instruction overriding a remembered style preference for the current turn;
- empty memory and Alexandria unavailable modes;
- long prompts/context trimming;
- hostile instruction inside a memory (treat as untrusted data);
- English, Indonesian, and mixed-language prompt token calibration;
- fast/deep mode separation and reasoning suppression;
- cancellation, Retry, and malformed/error terminal streams.

Do not use personal chat transcript samples for baseline evaluation.

Step 2 - Define performance budgets from measurements

Record TTFT, full completion time, cancellation latency, context assembly time, memory read time, error rate, and peak laptop memory for each mode. Set product thresholds based on the measured laptop/model combination rather than inventing a universal latency target. A release is not accepted if the spinner can remain forever, Stop fails to release the generation slot, or a network error silently creates a second generation.

Step 3 - Verify log and privacy boundaries

Use synthetic test markers and inspect gateway logs and Android logcat. Confirm none of the markers appears in logs; confirm no bearer token, full prompt, memory body, or thinking content appears. Ensure analytics/crash reporting is absent or explicitly configured to avoid user text and tokens. Confirm all chat history remains in Room and does not enter Alexandria unless a user deliberately invokes **Remember this** and confirms the memory.

R6 acceptance gate

- ☐ A compact set of 8-12 synthetic cases is recorded with mode/model/environment metadata.
- ☐ Memory-on/off comparison shows expected recall without irrelevant insertion.
- ☐ Token/context behavior is sanity-checked against the actual runtime; a large calibration corpus is optional.
- ☐ Cancellation, Retry, and network-loss behavior proven on device.
- ☐ Log inspection finds no test markers, prompts, memories, bearer tokens, or reasoning traces.
- ☐ Performance budgets are documented from measurements.

9. R7 - Release candidate and operations handover

Release build command sequence

After upgrading the toolchain and configuring signing:

```
.\gradlew.bat clean testDebugUnitTest lint assembleDebug assembleRelease --no-daemon
```

If device tests are added, include the appropriate `connected...AndroidTest` command on a configured emulator/device runner. Verify the release artifact signature and install the release build on the phone; a successful debug build is not a release gate.

Gateway pre-release sequence

```
python -m pip install -r requirements.txt
pytest -q
python -m friday_mobile_gateway.cli status
```

Then test the exact deployment configuration, actual Ollama models, Tailscale Serve URL, and token. Save a memory DB backup before any restore or migration experiment. Follow the Alexandria Production Blueprint and Runbook for database-specific restore testing.

Rollback procedure

1. Stop the gateway process before changing model/config or restoring SQLite.
2. Preserve the current release artifact and its source SHA.
3. For an app release, reinstall the previous signed build where Android package/version policy permits. Never tell users to uninstall if that would discard irreplaceable local chat history without a prior export/recovery decision.
4. For Alexandria data, restore only from a verified SQLite backup while all gateway/database connections are closed. Never replace a live WAL database file.
5. Restart, run health/auth/info tests, list approved memories, and send a synthetic chat request.
6. Record what was rolled back and which backup was used.

Production definition of done

The Must tier defines the release gate. Should items should be completed where they add direct value, but a large benchmark or major SDK migration is not a hidden blocker for a private sideload release.

Raphael is production-ready for the declared personal deployment only when **all** items below are true:

- ☐ Android CI green; Gateway CI green; Alexandria CI green for the release candidate.
- ☐ No known P0/P1 correctness or security defects remain open.
- ☐ Model/mode labels are truthful, selected tags are available, and no capability claim exceeds available evidence.
- ☐ Retry and stream persistence bugs are covered by regression tests.
- ☐ Release network policy is HTTPS-only; local debug exceptions are isolated.
- ☐ Token storage, backup exclusion, and erasure are tested; private gateway URL is user-configured.
- ☐ Room migrations and backup posture are tested.
- ☐ Tailscale-to-Ollama end-to-end checklist passes on the Xiaomi 12T.
- ☐ Logs are redacted and privacy behavior is verified with test markers.

- ☐ Signed release is built from a clean checkout and installed successfully; its stable signing key is backed up securely.
- ☐ Alexandria SQLite backup/restore runbook has been executed successfully at least once on a disposable database.
- ☐ README, docs/DONE.md, and evaluation report state only verified outcomes.
- ☐ Release version, source SHA, model tags, dependency/tool versions, artifact hash, and rollback instructions are recorded.

10. Reference commands and useful files

Gateway commands (Windows PowerShell)

```
# Start from repository root with the correct venv active
python -m friday_mobile_gateway.cli status
python -m uvicorn friday_mobile_gateway.app:app --host 127.0.0.1 --port 8765

# In another terminal
Invoke-RestMethod http://127.0.0.1:8765/v1/health
# After setting token in the environment, use it without echoing it to shared logs
Invoke-RestMethod http://127.0.0.1:8765/v1/info -Headers @{ Authorization = "Bearer $env:GATEWAY_BEARER_TOKEN" }

tailscale serve status
```

For local token-file configuration, remember rotate-token writes an ignored config.local.toml under src/friday_mobile_gateway. Restart Uvicorn after changing configuration.

Source-of-truth map

Concern	Authoritative location
Android UI and generation state	app/src/main/java/com/agri/raphael/ui/
Local conversation persistence	app/src/main/java/com/agri/raphael/data/local/
Android remote stream client	app/src/main/java/com/agri/raphael/inference/RemoteInferenceBackend.kt
Android token storage	app/src/main/java/com/agri/raphael/security/TokenStore.kt
Gateway config and defaults	src/friday_mobile_gateway/config.py, src/friday_mobile_gateway/config.toml
Gateway endpoints	src/friday_mobile_gateway/routes/
Provider and SSE	src/friday_mobile_gateway/provider_adapter.py, streaming.py
Gateway tests	tests/

Concern	Authoritative location
Embedded/shared memory sync	scripts/sync_alexandria.py, tests/test_alexandria_parity.py
Operator instructions	README.md, serve.ps1, serve.sh

11. Official references

References checked for this revision; re-check compatibility at release time:

1. Google Play target API level requirements: https://support.google.com/googleplay/android-developer/answer/11926878
2. Android Gradle Plugin compatibility and API support: https://developer.android.com/build/releases/about-agp
3. Android Network Security Configuration: https://developer.android.com/privacy-and-security/security-config
4. AndroidX EncryptedSharedPreferences API and deprecation notice: https://developer.android.com/reference/androidx/security/crypto/EncryptedSharedPreferences
5. Ollama chat API: https://docs.ollama.com/api/chat
6. Tailscale Serve: https://tailscale.com/docs/features/tailscale-serve
7. Android architecture recommendations: https://developer.android.com/topic/architecture/recommendations
8. Python release lifecycle and active versions: https://www.python.org/downloads/
9. SQLite partial indexes: https://www.sqlite.org/partialindex.html

Canonical roadmap reminder: do R0, then R1, then R2 through R7. Feature polish is deliberately last. Revision 3.4 is the current roadmap. Markdown is the source of truth; DOCX/PDF copies are generated from this document.